End-to-End Modern Data Pipeline & Analytics Platform

Local Deployment Edition

Data Engineering — Trabajo Práctico 2026

Your engineering objective is to design, implement, and validate an end-to-end modern data architecture running entirely on your local machine, orchestrated by Apache Airflow. The pipeline will ingest raw multi-table data, enforce data hygiene and PII governance, and curate an analytical storage layer optimized for high-performance SQL queries using DuckDB.

The architecture must follow production engineering discipline: your pipeline must be fully parameterized, deterministic, and idempotent. Your storage assets, schemas, and pipeline code must be permanently reproducible and version-controlled.

1. Dataset Selection & Requirements

You may bring your own dataset or choose one of the pre-approved options below. If selecting your own dataset, it must satisfy the following technical criteria:

- **Size:** Uncompressed volume between 50 MB and 300 MB (sufficient to demonstrate partition pruning on a local filesystem).
- **Relational Complexity:** A minimum of three distinct, joinable tables/entities (e.g., Transactions, Entities/Users, Products/Catalogs).
- **Temporal Column:** At least one native date/timestamp field (e.g., `transaction_timestamp`, `event_date`) to drive logical partitioning and backfills.
- **Data Hygiene Challenges:** The raw data must exhibit real-world imperfections (untyped strings for dates/numbers, nullable fields requiring business rules, inconsistent casing).
- **PII Target:** If your raw dataset lacks direct PII, you must generate and attach a synthetic identity table (e.g., `full_name`, `email`, `phone_number`) mapped to your primary entity ID prior to Bronze ingestion.

Pre-Approved Dataset Options

1. **Brazilian E-Commerce (Olist):** 9 relational tables covering orders, payments, reviews, and items (~100 MB). Requires generating a synthetic customer identity table with name and email mapped to `customer_id`. Link: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
2. **Steam Games & User Reviews:** Relational metadata linked to structured user reviews and engagement metrics (~110 MB). Requires generating a synthetic gamer profile table containing usernames and email addresses. Link: https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam
3. **Formula 1 World Championship (1950–2024):** 14 relational tables detailing circuits, races, drivers, and results (~65 MB). Note: Because this sits at the lower end of the size requirements, the partition pruning

demonstration will be less dramatic than the Olist or Steam datasets. Link: https://www.kaggle.com/datasets/rohanrao/formula1-world-championship-1950-2020

2. Architecture & Pipeline Specifications

The pipeline follows the Medallion Architecture pattern with three layers stored on the local filesystem, processed by Python tasks orchestrated by Airflow, and queried via DuckDB.

Local Storage Layout

All data layers are stored under the Airflow home directory. The following folder structure must be created before pipeline execution:

```
~/airflow/data/
├── bronze/
│   └── {table}/
│       └── ingested_year=YYYY/
│           └── ingested_month=MM/
├── silver/
│   └── {table}/
└── gold/
    └── {dataset}/
        └── year=YYYY/
            └── month=MM/
```

Bronze Layer (Raw Ingestion)

- The Airflow DAG must extract the source tables and land them into dedicated subfolders encoding the ingestion lineage in the path: `~/airflow/data/bronze/{table}/ingested_year=YYYY/ingested_month=MM/`
- Data must remain in its native raw format (CSV or JSON) without modifications.
- **Ingestion vs. Business Date:** Understand the distinction between the ingestion date (when the file was loaded) and the business date (the date the event occurred in the source system).

Silver Layer (Hygiene, Typing, and Governance)

- Transform Bronze assets using **pandas + pyarrow** inside Airflow `@task` decorated functions. Do not use external compute engines.
- **Deduplication:** The Silver layer must resolve duplicate records from the source using programmatic deduplication, retaining only the most recent or canonical version of each entity.
- **Data Typing & Hygiene:** Convert string representations of dates into native `datetime` types; cast numeric strings to appropriate numeric types; address orphan records and null anomalies through explicit business logic.
    - *Documentation Rule:* Document each null-handling decision in your README.md with its business justification — e.g., "rows with null `order_id` are dropped because they cannot be joined to any downstream table" or "null `delivery_date` values are imputed with a sentinel date representing an active shipment."
- **Verifiable PII Governance:** Direct identifier fields (such as email) must be anonymized using cryptographic hashing (e.g., SHA-256) combined with an application-level salt so analysts can join on the hash without recovering the original string.

    - *Implementation Rule:* Store the application salt as an Airflow Variable via the CLI: `airflow variables set PII_SALT <your-secret-value>`
    - *Airflow Best Practice:* Do not call `Variable.get('PII_SALT')` at the top level (module level) of your DAG file, as this hammers the metadata database every 30 seconds during DAG parsing. Retrieve the salt inside the task callable (inside your `@task` decorated function) or via Jinja templating (`{{ var.value.pii_salt }}`). It must never appear in any committed file or repository script.
- **Storage Format:** Stored strictly as Apache Parquet using pyarrow.

Gold Layer (Curated Analytical Model)

- **Join Complexity:** The Gold dataset must incorporate data joined from at least **four distinct Silver entities** (e.g., orders + payments + order items + products, or races + results + drivers + constructors).
- **Hive-Style Partitioning:** The Gold dataset must be written using explicit temporal partitioning based on your primary business date field: `~/airflow/data/gold/{dataset}/year=YYYY/month=MM/`
- **Local Catalog:** Expose the Gold layer to DuckDB. DuckDB can infer the schema and partitions automatically using: `SELECT * FROM read_parquet('~/airflow/data/gold/{dataset}/**/*.parquet', hive_partitioning=True)`

3. Core Engineering Requirements: Orchestration & Idempotency

Task Granularity & DAG Composition

- **Minimum Structure:** Your DAG must contain at least **five distinct tasks** with explicit >> dependencies between them (e.g., landing raw files >> validating Bronze >> cleaning/typing Silver >> masking PII >> generating Gold partitions >> updating local catalog).

Deterministic Execution & Idempotency

- Your DAG must accept an execution date context (`{{ data_interval_start }}` or `{{ ds }}`) and process data strictly within that logical time window.
- **The Idempotency Test:** Running the DAG twice for the exact same execution date or month must result in the exact same record count in Silver and Gold. You must demonstrate this in your submission.
- Implement an explicit write strategy to guarantee that manual re-runs or retries do not append duplicate rows. For local Parquet files, this means **overwriting the partition folder** on each run rather than appending to existing files.

4. DuckDB Local Query Validation (FinOps Equivalent)

In production, cloud data warehouses like Amazon Athena charge per terabyte scanned — making partition pruning a financial imperative. In our local setup, we use DuckDB to validate the same engineering principle: that your Hive-style partitioning actually causes the query engine to skip irrelevant files.

1. **Full Scan Query:** Execute a representative aggregation query across the entire Gold dataset without specifying partition filters, using DuckDB's `EXPLAIN ANALYZE`.

2. **Pruned Query:** Execute the identical query adding a `WHERE year = '...' AND month = '...'` filter targeting a single partition.
3. **Baseline Expectation:** The pruned query must read fewer Parquet files than the full scan. If DuckDB reads the same number of files in both cases, your partitioning strategy is not working — revisit your folder structure.
4. **Deliverable:** Provide the terminal output of both `EXPLAIN ANALYZE` statements side by side, showing the difference in **"Rows Read"** and implicitly in files accessed.

Example DuckDB validation script (`scripts/finops_validation.py`):

```
import duckdb

DATA_PATH = "~/airflow/data/gold/{dataset}/**/*.parquet"

conn = duckdb.connect()

print("=== FULL SCAN (no partition filter) ===")
conn.execute(f"""
    EXPLAIN ANALYZE
    SELECT year, month, COUNT(*) as records
    FROM read_parquet('{DATA_PATH}', hive_partitioning=True)
    GROUP BY year, month
""").df().pipe(print)

print("\n=== PRUNED SCAN (single partition) ===")
conn.execute(f"""
    EXPLAIN ANALYZE
    SELECT year, month, COUNT(*) as records
    FROM read_parquet('{DATA_PATH}', hive_partitioning=True)
    WHERE year = '2017' AND month = '01'
    GROUP BY year, month
""").df().pipe(print)
```

5. Local Airflow Installation

Airflow runs on your local machine using `airflow standalone`, which starts the webserver and scheduler in a single process — appropriate for development and demonstration purposes.

Installation Steps

1. Enable WSL2 on Windows and install Ubuntu: `wsl --install -d Ubuntu`
2. Inside Ubuntu, install Python 3.12 and build tools:
```
sudo add-apt-repository ppa:deadsnakes/ppa -y && sudo apt update && sudo apt
install -y python3.12 python3.12-venv python3.12-dev build-essential
```
3. Create a virtual environment:
```
python3.12 -m venv ~/airflow-env && source ~/airflow-env/bin/activate
```
4. Install Airflow 3.0.4:
```
pip install "apache-airflow==3.0.4" --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-3.0.4/constraints-
3.12.txt"
```
5. Initialize the database: `airflow db migrate`
6. Start Airflow: `airflow standalone`

7. Open the UI at `http://localhost:8080` (credentials printed in the terminal)

Starting Airflow After Each Reboot

Every time you restart your machine, open the Ubuntu terminal and run:

```
source ~/airflow-env/bin/activate
airflow standalone
```

The admin password is stable between restarts. To retrieve it: `cat ~/airflow/standalone_admin_password.txt`

DAG Development

Install the VS Code WSL extension (`ms-vscode-remote.remote-wsl`), then from the Ubuntu terminal open your DAGs folder directly in VS Code:

```
mkdir -p ~/airflow/dags
cd ~/airflow/dags
code .
```

Any `.py` file saved in `~/airflow/dags/` is automatically detected by Airflow within ~30 seconds.

6. Repository & Version Control Discipline

All project assets must be maintained in a clean, version-controlled repository (GitHub or GitLab):

- **Structure:** Organize into `/dags`, `/scripts`, `/sql`, and `/docs`.
- **.gitignore:** Your local data directory (`~/airflow/data/`) must NOT be committed to the repository — only the pipeline code that generates it. Add any synthetic PII CSV files to `.gitignore`.
- **Branch protection:** Use a `main` (production) branch protected from direct commits, and a `develop` branch for integration. Feature work goes on `feature/` branches merged via Pull Request.

7. Defense Preparation

Each team will have to trigger an execution of their pipeline, show their DuckDB query validation output, and defend their design. Example questions include:

- **Idempotency:** Show me what happens in your Silver and Gold layers when you manually trigger a re-run of a previously successful month. Walk me through the code that prevents duplicate records. How does overwriting the partition folder guarantee idempotency?
- **Task Composition:** Walk through your DAG graph view. Why did you separate these specific transformation steps into distinct Airflow tasks instead of combining them?
- **Data Hygiene:** Open your README.md and explain your handling of nulls in table X. Why was dropping/imputing those records the correct business decision over the alternative? How did you ensure true deduplication of source records?
- **Partitioning & Query Performance:** Why did you choose your specific partition key over other candidates? Show me the DuckDB EXPLAIN ANALYZE output proving that the pruned query reads fewer

files than the full scan. What is the difference between your Bronze ingestion date and your Gold business date?

- **Security & Governance:** Demonstrate how you retrieve your PII salt inside the DAG. If an analyst gains read access to the Silver Parquet files on disk, what prevents them from reversing the hashed emails? Why is the salt stored as an Airflow Variable instead of a Python constant?
- **Data Modeling:** Explain the joining logic used to construct your Gold table. Why did you choose to denormalize these specific four entities rather than leaving them in third normal form?
- **Local vs. Cloud:** How would you migrate this pipeline to AWS if the organization decided to scale? Which components would map to which AWS services, and what code changes would be required?